Practo (Connecting patients to doctors/hospitals using AWS) - Week 1

Diya Patel (202301216), Gautam Modi (202301214), Jevik Bhalodiya (202303021)
IT457 - Cloud Computing, Dhirubhai Ambani University

I. Introduction

Practo is a Bangalore-based healthcare platform [1]. It serves two sides of the market: patients, who search for doctors and book appointments, and providers, who run their practices on Practo's software (Practo Ray) [2]. In May 2016 Bloomberg reported 60 million users, 200,000 doctors, 10,000 hospitals and over 40 million appointments a year [1], and the founder gives a similar appointment volume [3]. An AWS India executive has also cited Practo as an example AWS customer [4]. These are older figures, not current counts. This report covers the problems Practo faced, how its architecture on AWS evolved, the salient numbers, and how they connect to our exercise.

II. Problems and Bottlenecks

Practo's engineers describe a monolith on AWS Auto Scaling that was later split into services [5]. The first bottleneck was organisational: as the number of services grew, developers depended on the infrastructure team to provision each new service [5]. The second was cost: running a separate Auto Scaling group for every service left capacity underused [5]. Practo also needed to separate user-facing transactional requests from asynchronous background work, so that jobs did not require always-on capacity [5].

The database layer presented a further challenge. Practo's Amazon RDS databases ran MySQL 5.1, auto-generated SQL queries were slow, and disk I/O and temporary-table bottlenecks hurt performance under heavy transactional load. This slowed searches and bookings at peak hours while RDS costs kept rising [6].

III. Evolution of the Architecture

The speaker describes two stages: a monolith on AWS Auto Scaling, then a service-oriented design that now runs on Kubernetes with close to 80 services in production [5]. In parallel, the database was tuned with provisioned IOPS, upgrades from MySQL 5.1 to 5.5 and then 5.6, rewritten queries, cleaner schemas and indexes, and online schema changes built into the deployment pipeline [6]. Looking ahead, the speaker names a service mesh and Amazon EKS, so that Kubernetes management can be handed to AWS [5]. The sources do not date the database tuning, so we cannot place it exactly within these stages.

IV. Architecture

The architecture below combines a Practo engineer's walk-through [5] with Practo's public code [7], [8]. It is our reading of those sources, not an official diagram, and no AWS regions are published.

i) Request path: A user request reaches an Auto Scaling group behind a load balancer, which forwards it to NGINX router machines. These pass it to a second load balancer in front of the Kubernetes cluster, which is managed with kops and runs the services [5].

ii) Developer workflow: Each service has a Dockerfile in its codebase, and the CI/CD pipeline builds an image on every push for local, staging and production use. An in-house domain-specific language lets developers declare supporting infrastructure such as RDS, Memcached and Redis without waiting on the infrastructure team [5].

iii) Background jobs: Services place jobs in Amazon SQS, and worker pods consume them on cheaper spot instances. Practo's Worker Pod Autoscaler scales the workers from queue metrics [5], [7]. Its documentation explains that queue length alone can mislead, because fast workers keep the backlog near zero, so it also considers message activity and processing time, and it allows a minimum of zero workers [7].

iv) Monitoring and data: Load-balancer logs go to S3, request logs go to Graylog with alerts, and errors go to Sentry [5]. A separate pipeline, Tipoca Stream, is described as a near-real-time RDS-to-Redshift pipeline with masking and reloading support [8].

v) Database layer: RDS MySQL tuned as described above [6].

V. Key Numbers Reported

Table I lists the main figures we found, each with the source it comes from and why it matters for our exercise.

VI. Criticisms and Limitations

First, the strongest cost and performance claims are self-reported: the 40% saving comes from the speaker [5], and the $5\times$ gain comes from a vendor's promotional page that gives no benchmark or date [6]. Second, The sources cover different periods and components and therefore do not describe a single architecture from the same period. We found no official AWS case-study page for Practo, and no spend, uptime or latency data.

Third, the design carries operational weight. Practo runs self-managed Kubernetes with in-house tools such as its

TABLE I
KEY REPORTED NUMBERS FOR PRACTO ON AWS

Metric	Reported value	Relevance to our exercise	Ref.
Users, doctors, hospitals	60M, 200K, 10K (May 2016)	Sizes the doctor dataset we seed in DynamoDB	[1]
Appointments per year	Over 40M	About 1.3 bookings per second on average (our calculation); baseline for our load test	[1], [3]
Services in production	About 80	Contrast with our small serverless design	[5]
Infrastructure cost saving	Over 40% from one shared Kubernetes cluster, reported	Baseline for our cluster vs serverless cost comparison	[5]
Database performance	5× faster, reported; lower monthly RDS bills	Motivates measuring database latency and cost	[6]
Company-page claims	1 lakh+ doctors, 30 crore+ patients, 20+ countries (undated)	Shows current claims differ from 2016 figures	[9]

Note: All figures are self-reported by Practo, a vendor or the press, and none has been independently audited. The 2016 figures measure reach, not peak load. The 40% saving and the 5× gain come with no baseline spend or benchmark.

autoscaler and infrastructure language, and it plans EKS to offload Kubernetes management [5].

Fourth, the sources disagree on details: the founding year is 2008 in one report [1] and 2009 in another [10], and a 2015 report gives 8,000 hospitals against 10,000 elsewhere [1], [2]. The company page lists 1 lakh+ doctors against 200,000 in older sources [9].

Finally, the AWS testimonials page [11] includes a Practo remark attributed to Siddhartha Nihalani, Director of Engineering, describing how AWS influenced application design and reliability through service-level backup and failover capabilities. We read the French version, which appears to have a 2021 footer; we have not independently confirmed the quote against the English original. The testimonial provides qualitative evidence of Practo's approach to reliability, but no quantitative uptime measurements. The *67 billion events* AI/ML claim in our topic brief belongs to Shaadi.com, not Practo.

VII. LINK TO OUR EXERCISE

We propose to build a Practo-style doctor-search and booking API using AWS Lambda, API Gateway and DynamoDB. We will measure cold-start latency, request latency, scaling behaviour, error rate and cost per 1,000 requests, then compare the serverless cost model with the reported always-on cluster approach.

REFERENCES

[1] Bloomberg, "India's Practo plans to take digital healthcare success abroad," May 27, 2016. [Online]. Available: https://www.bloomberg.com/news/articles/2016-05-27/india-s-practo-plans-to-take-digital-health-care-success-abroad

[2] TechCrunch, "Indian health search portal Practo lands $90M led by Tencent for global expansion," Aug. 6, 2015. [Online]. Available: https://techcrunch.com/2015/08/06/indian-health-search-portal-practo-lands-90m-led-by-tencent-for-global-expansion

[3] DQ India, "Practo is the world's largest healthcare appointment booking platform: Shashank ND, Founder, Practo." [Online]. Available: https://www.dqindia.com/practo-is-the-worlds-largest-healthcare-appointment-booking-platform-shashank-nd-founder-practo/

[4] CXOToday, "How AWS is driving India's cloud growth story." [Online]. Available: https://cxotoday.com/cloud/aws-and-cloud-growth-to-accelerate/

[5] Amazon Web Services, "This Is My Architecture: Practo," YouTube video. [Online]. Available: https://youtu.be/YgABcxC4pwQ

[6] Mydbops, "Practo AWS RDS database optimization case study." [Online]. Available: https://www.mydbops.com/case-studies/practo-database-optimization-aws-rds (accessed Oct. 21, 2026).

[7] Practo, "k8s-worker-pod-autoscaler," GitHub. [Online]. Available: https://github.com/practo/k8s-worker-pod-autoscaler (accessed Oct. 21, 2026).

[8] Practo, GitHub organization (Tipoca Stream). [Online]. Available: https://github.com/practo (accessed Oct. 21, 2026).

[9] Practo, "Practo for Providers." [Online]. Available: https://www.practo.com/providers (accessed Oct. 21, 2026).

[10] BW Disrupt, "Practo aims to make healthcare a delightful experience." [Online]. Available: https://www.bwdisrupt.com/article/practo-aims-to-make-healthcare-a-delightful-experience-104958

[11] Amazon Web Services, "Customer testimonials," French version. [Online]. Available: https://aws.amazon.com/fr/compliance/testimonials1/